ResQLink

Real-Time Emergency Resource Coordination & Availability Platform

FINAL-YEAR B.TECH PROJECT BLUEPRINT

1. Executive Summary

ResQLink is a real-time coordination platform for organizations and people who need to discover, request, reserve, and track emergency resources. It addresses fragmented resource information across calls, spreadsheets, messages, and separate organizations.

It is an information and coordination layer, not a replacement for emergency services or a medical decision system.

2. Problem Statement

• Emergency resource information is distributed across multiple channels.

• Requesters may not know which participating provider currently has capacity.

• Availability changes quickly, making stale information operationally risky.

• Repeated requests can create duplicate work.

• Organizations need a central history of requests, assignments, status changes, and response times.

3. Proposed Solution

A centralized event-driven platform where participating organizations publish resource availability and authorized users create resource requests. A matching engine finds relevant resources, Redis maintains fast-changing state, RabbitMQ handles asynchronous events, PostgreSQL stores durable records, and React provides dashboards.

4. Scope

4.1 MVP

• Registration/login

• Role-based access: requester, provider, coordinator, administrator

• Organization and resource management

• Resource request creation and status tracking

• Matching by resource type, area, availability and urgency

• Reservation preventing double allocation

• In-app notifications

• Operational analytics

• Audit log

• Docker deployment on Linux server

• Jenkins CI/CD

4.2 Future Scope

• Mobile app

• Advanced geospatial search

• Demand forecasting

• Advanced duplicate-incident detection

• SMS/WhatsApp integrations through approved providers

• Multi-region/Kubernetes deployment

5. Users and Roles

Role	Responsibilities
Requester	Create requests, view status, accept/confirm matches, history.
Resource Provider	Register resources, update availability, accept/reserve requests.
Coordinator	Monitor active requests, resolve conflicts, oversee assignments.
Administrator	Manage users, organizations, policies, audit and configuration.

6. Real-Life Workflow

• 1. Provider registers a resource.

• 2. Requester creates a request with type, urgency and location.

• 3. FastAPI validates and stores it in PostgreSQL.

• 4. FastAPI publishes request.created to RabbitMQ.

• 5. Matching worker checks candidate resources.

• 6. Redis provides current availability; PostgreSQL provides durable details.

• 7. System creates match candidates and notifications.

• 8. Provider reserves a resource.

• 9. Database transaction prevents double allocation.

• 10. Request ends as RESOLVED or CANCELLED with status history.

7. High-Level Architecture

Recommended first release: modular monolith + background workers. This is simpler than many microservices while retaining event-driven scalability.

Flow: Client → Nginx → React/FastAPI → PostgreSQL + Redis + RabbitMQ → Workers

8. Architecture Blueprint

```
USER / ORGANIZATION
|
v
React + Vite Frontend
|
HTTPS
|
Nginx Reverse Proxy
|
Python FastAPI API
| |
v v
PostgreSQL Redis
|
```

```
v
RabbitMQ
| |
v v
Matching Notification/
Worker Analytics Workers

GitHub → Jenkins → Test → Docker → Deploy → Linux Server
```

9. Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Interactive dashboards.
UI	Tailwind CSS	Responsive interface.
API	Python + FastAPI	REST APIs, validation, business logic.
ORM	SQLAlchemy	Models and transactions.
Database	PostgreSQL	Durable relational data.
Cache/State	Redis	Live state, cache, rate limiting.
Messaging	RabbitMQ	Async events/background jobs.
Workers	Python	Matching, notifications, analytics.
Auth	JWT + password hashing	Authentication/authorization.
Proxy	Nginx	HTTPS/routing.
Containers	Docker + Compose	Reproducible deployment.
CI/CD	Jenkins	Automated test/build/deploy.
VCS	Git + GitHub	Source control.
Server	Linux VPS/cloud VM	Production hosting.

10. Database Design

Table	Important Fields	Purpose
users	id, name, email, password_hash, role	Authentication/roles.
organizations	id, name, type, address, status	Participating organizations.
resources	id, organization_id, type, status	Available resources.
resource_availability	resource_id, status, start_at, end_at	Availability.
requests	id, requester_id, type, urgency, location, status	Requests.
matches	id, request_id, resource_id, score, status	Matches.
reservations	id, request_id, resource_id, expires_at	Allocation.
notifications	id, user_id, type, message, read_at	Notifications.
status_history	entity_type, entity_id, old_status, new_status	Audit trail.
audit_logs	actor_id, action, entity_type, entity_id	Security/operations.

11. State Machines

Resource: AVAILABLE → RESERVED → BUSY/IN_USE → AVAILABLE. Reservation may return to AVAILABLE on expiry/cancellation. OFFLINE is provider-controlled.

Request: OPEN → MATCHED → RESERVED → IN_PROGRESS → RESOLVED, or CANCELLED. Failed/expired matches may return the request to OPEN.

12. Matching Engine

Start with deterministic and explainable matching, not machine learning.

Factor	Use
Resource type	Hard requirement.
Availability	Hard requirement.
Area/distance	Higher score for closer candidates when location exists.
Urgency	Higher priority for critical requests.
Capacity	Must satisfy required capacity.
Eligibility	Only approved providers.

The UI should explain why a candidate matched. Weights should be configurable.

13. Redis Design

• Cache frequently accessed resource/organization data.

• Maintain short-lived live availability/state.

• Use TTLs for temporary keys.

• Rate-limit selected APIs.

• Use short-lived locks only where necessary.

• Never use Redis as the permanent source of truth.

14. RabbitMQ Design

Event	Producer	Consumer
request.created	FastAPI	Matching worker
match.created	Matching worker	Notification worker
reservation.created	FastAPI	Notification/audit worker
reservation.expired	Expiry worker	Resource state worker
request.resolved	FastAPI	Analytics/notification worker

Use durable queues, acknowledgements, retries and dead-letter handling. Workers should be idempotent where practical.

15. API Blueprint

Method	Endpoint	Purpose
POST	/api/auth/register	Register
POST	/api/auth/login	Login
GET	/api/resources	Search
POST	/api/resources	Create
PATCH	/api/resources/{id}/status	Change status
POST	/api/requests	Create request

Method	Endpoint	Purpose
GET	/api/requests	List requests
GET	/api/requests/{id}	Details
POST	/api/requests/{id}/match	Match
POST	/api/reservations	Reserve
POST	/api/reservations/{id}/cancel	Cancel
GET	/api/notifications	Notifications
GET	/api/dashboard/metrics	Metrics

16. Frontend Screens

Screen	Purpose
Landing	Platform overview.
Login/Register	Authentication.
Requester Dashboard	Requests, matches, status.
Create Request	Request form.
Resource Search	Eligible resources.
Provider Dashboard	Resources and incoming requests.
Resource Management	Update resource status.
Coordinator Dashboard	Active requests and assignments.
Notifications	Updates.
History	Past activity.
Admin Panel	Users, organizations, audit.

17. Security

• Strong password hashing; never plain text.

• JWT expiration.

• Backend-enforced RBAC.

• Pydantic/input validation.

• ORM/parameterized database operations.

• Rate limiting.

• Audit logs.

• HTTPS.

• Secrets in environment/Jenkins credentials.

• Least-privilege accounts.

18. Concurrency & Reliability

Prevent double reservation with PostgreSQL transactions, locking and constraints. PostgreSQL remains authoritative for durable allocation.

• RabbitMQ acknowledgements after successful processing.

• Retry transient worker failures.

• Dead-letter queue for repeated failures.

• Idempotency for critical operations where appropriate.

19. Jenkins CI/CD

```
Developer → GitHub Push → Jenkins → Checkout → Lint → Test → React Build → Docker Build → Deploy →
Health Check → Production
```

• Keep a Jenkinsfile in the repository.

• Store production secrets in Jenkins credentials.

• Deploy from the selected protected branch.

20. Server Deployment

• Linux VPS/cloud VM.

• Docker Compose for application dependencies.

• Nginx reverse proxy.

• HTTPS certificate.

• Firewall with required public ports only.

• Persistent volumes and PostgreSQL backups.

• Health endpoint such as /health.

• Structured logs.

21. Docker Services

Container	Role
frontend	React production build
api	FastAPI
worker	Matching/notification jobs
postgres	Primary DB
redis	Cache/live state
rabbitmq	Message broker
nginx	Reverse proxy
jenkins	CI/CD, preferably isolated

22. Observability

• Structured logs with request IDs.

• Health checks.

• Track latency, worker failures, queue depth and errors.

• Keep business metrics separate from raw logs.

23. Non-Functional Requirements

Requirement	MVP Target
Availability	Clean recovery after restart.
Performance	Common APIs normally respond within a few hundred ms under normal demo load.
Scalability	API/workers can scale horizontally.
Security	Auth, authorization, validation, HTTPS, secret management.
Reliability	Retries/dead-letter for queued jobs.
Maintainability	Modular backend, tests, docs.
Deployability	Docker Compose + Jenkins deployment.

24. Project Structure

```
resqlink/
■■■ frontend/src/{components,pages,services,hooks,utils}
■■■ backend/app/{api,core,models,schemas,services,workers,main.py}
■■■ backend/tests/
■■■ infrastructure/{docker-compose.yml,nginx,scripts}
■■■ Jenkinsfile
■■■ .env.example
■■■ README.md
■■■ docs/
```

25. Development Strategy

Build in layers: core database/API → authentication → resources → requests/reservations → matching → Redis → RabbitMQ workers → notifications/dashboards → tests/security → Docker/server → Jenkins → documentation/demo.

26. Why This Project Is Meaningful

• Solves coordination/visibility problems rather than only CRUD.

• Dynamic availability justifies Redis.

• Asynchronous work justifies RabbitMQ.

• Transactional records justify PostgreSQL.

• API/workers can scale horizontally.

• Can be demonstrated with simulated data without sensitive real emergency data.

27. What Not to Build in V1

• National emergency network.

• Medical diagnosis/treatment advice.

• Autonomous emergency dispatch.

• Microservice for every table.

• Mobile app before web MVP.

• AI just for the label.

• Real sensitive emergency/medical data.

28. Final Demo Scenario

Use simulated organizations/resources. Demonstrate: provider makes a resource available → requester creates a high-priority request → FastAPI persists it → RabbitMQ triggers matching/notification workers → Redis reflects live state → provider reserves resource → requester sees updated status → Jenkins deploys the application.

29. Viva Talking Points

• Why PostgreSQL? Durable relational source of truth and transactions.

• Why Redis? Fast changing state, cache and rate limiting.

• Why RabbitMQ? Decouples background work from API requests.

• Why FastAPI? Lightweight Python API framework with validation.

• Why React? Component-based interactive dashboards.

• Why Docker? Reproducible environments.

• Why Jenkins? Automated build/test/deploy.

• Why not microservices immediately? Modular monolith + workers is simpler and sufficient for the MVP.

• How prevent double reservation? Database transactions/locking/constraints.

• What if RabbitMQ fails? Retry + dead-letter handling.

30. Success Criteria

• Authentication works.

• Providers manage resources and availability.

• Requesters create requests.

• Eligible resources are matched.

• Double reservation is prevented.

• RabbitMQ handles a real asynchronous workflow.

• Redis is used for live/cache data.

• Status history is recorded.

• React dashboards work.

• Tests run in Jenkins.

• Jenkins deploys to server.

• Production app is accessible over HTTPS.

31. Final Blueprint

ResQLink = React + FastAPI + PostgreSQL + Redis + RabbitMQ + Python Workers + Docker + Jenkins + Linux Server — a real-time resource coordination platform with transactional allocation, asynchronous events, live availability, matching, notifications, and scalable deployment.

Blueprint version 1.0 — MVP first, scalability second